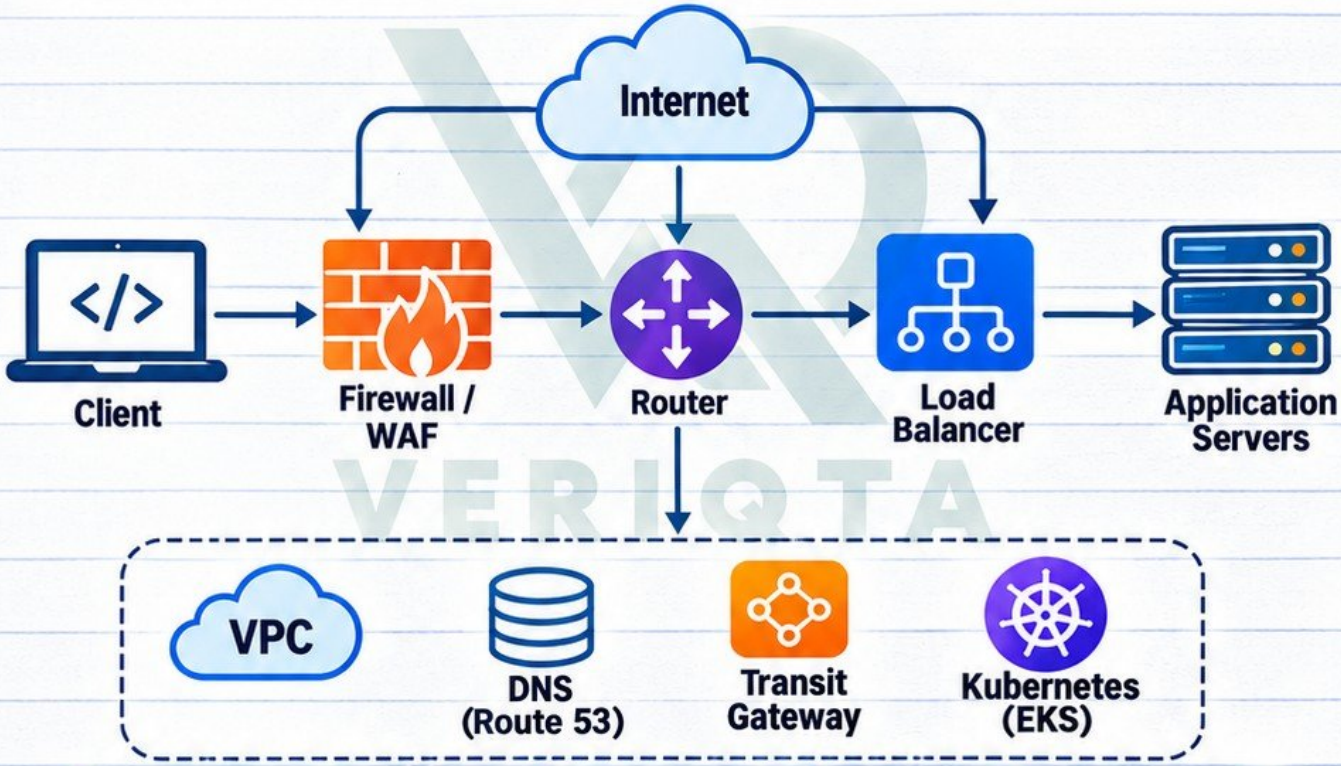
VERIQTA

50 Networking Problems for DevOps Engineers

Real Production Incidents, Root Causes and Fixes



- Connectivity Failures
- DNS & Name Resolution Issues
- Routing and Traffic Problems
- Load Balancing Issues
- Kubernetes Networking Problems
- Real Troubleshooting and Fixes

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

VERIQTA

The Network Is Up, but Production Requests Keep Disappearing

1 TCP connections silently fail when the Linux conntrack table fills.

What Happens

- New connections are dropped or not tracked.
- Existing sessions may break intermittently.
- Application errors increase even though the network seems healthy.



2 Ephemeral port exhaustion breaks outbound API calls despite healthy CPU and memory.

What Happens

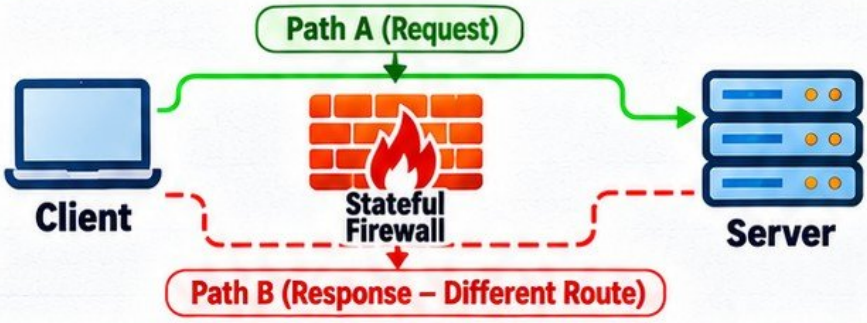
- The host runs out of available ephemeral ports.
- New outbound connections (e.g., to external APIs) fail.
- CPU and memory remain normal, making it misleading to diagnose.



3 Asymmetric routing causes established sessions to disappear behind stateful firewalls.

What Happens

- Traffic takes different paths in and out.
- The firewall does not see both sides of the connection.
- Valid sessions are dropped because the state is not found.

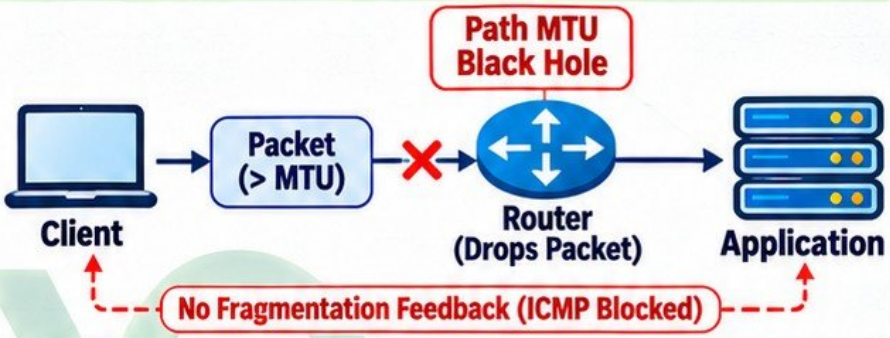


The Packet That Never Reached the Application

1 Path MTU black holes break TLS handshakes and large HTTP responses.

What Happens

- Packets larger than the path MTU are dropped.
- ICMP "Fragmentation Needed" messages are blocked.
- TLS handshakes or large responses time out.



2 TCP retransmissions create extreme latency without complete packet loss.

What Happens

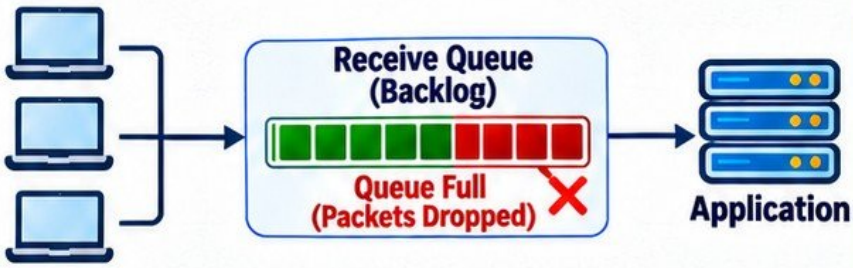
- Some packets are lost or delayed in transit.
- TCP retransmits the missing packets.
- The connection stays alive, but latency increases significantly.



3 Receive queues overflow during traffic bursts while network bandwidth remains available.

What Happens

- Inbound packets arrive faster than the application can process.
- The kernel's receive queue (softnet backlog) fills up.
- Legitimate packets are dropped even though the network link has free capacity.



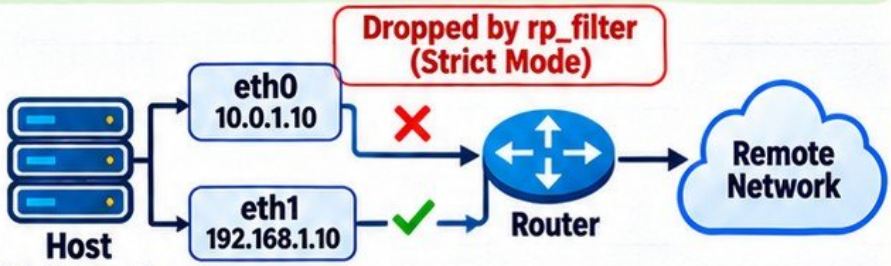
VERIQTA

When Linux Network Tuning Makes Production Worse

1 Incorrect reverse-path filtering drops legitimate multi-homed traffic.

What Happens

- rp_filter is too strict (e.g., 1 or 2) and drops valid traffic from a second interface.
- A host with multiple networks (e.g., public and private) loses legitimate responses.
- Connections fail intermittently, especially after routing changes.

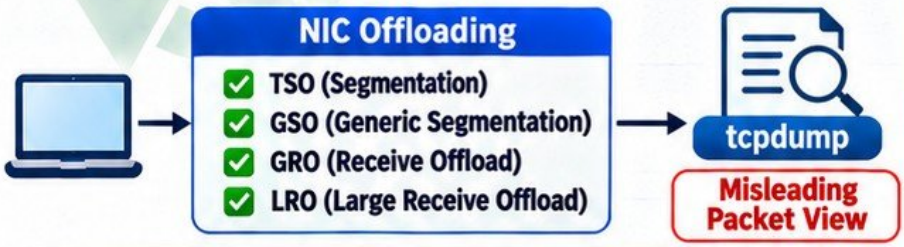


Check: `sysctl net.ipv4.conf.all.rp_filter`

2 Network interface offloading produces misleading packet captures and complicates diagnosis.

What Happens

- Features like TSO, GSO, GRO, and LRO change how packets are handled.
- Packet captures (tcpdump) may show large or merged packets that never existed on the wire.
- This can lead to false conclusions during troubleshooting.

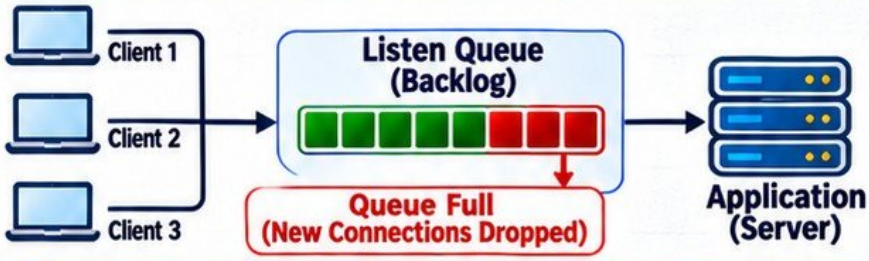


Check: `ethtool -k eth0` | Fix: `ethtool -K eth0 gro off gso off`

3 TCP backlog exhaustion causes intermittent connection failures under load.

What Happens

- The listen queue (backlog) fills up when connection requests arrive faster than the app can accept.
- New connections are dropped, leading to timeouts or "connection refused" errors.
- Often appears only during traffic spikes or high concurrency.



Check: `ss -ltn` | Tune: `sysctl -w net.core.somaxconn=4096`

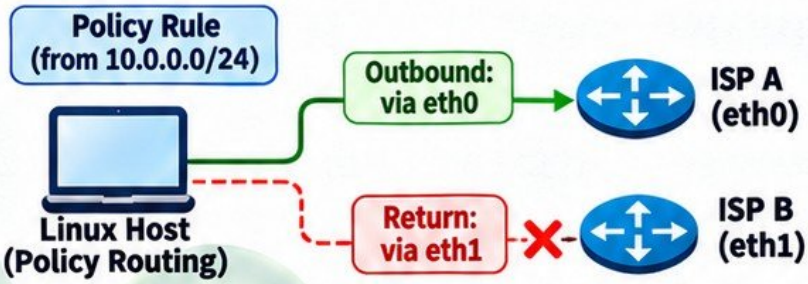
VERIQTA

The Routing Table That Sent Traffic Into a Black Hole

1 Policy-based routing sends return packets through the wrong interface.

What Happens

- Traffic matches a policy rule and leaves via a different interface.
- The return path does not match, so responses are lost.
- Connections time out even though the destination is reachable.

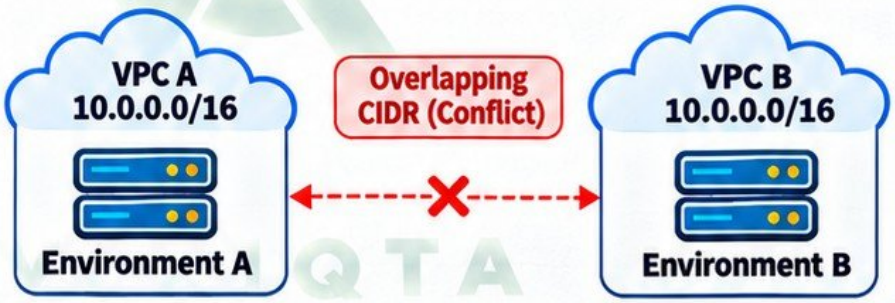



```
Check: ip rule show | ip route show table all
```

2 Overlapping private CIDR ranges break connectivity between environments.

What Happens

- Two environments use the same private IP range (e.g., 10.0.0.0/16).
- Routes cannot uniquely determine the destination.
- Traffic is sent to the wrong network, causing timeouts or unexpected access.



3 A default-route change disrupts production traffic after a VPN failover.

What Happens

- During VPN failover, a new default route is added or preferred.
- Production traffic is sent to the wrong gateway (e.g., VPN instead of the internet).
- External services become unreachable, causing outages.

Routing Table (After Failover)

Destination	Next Hop	Interface
0.0.0.0/0	10.10.0.1	tun0 (VPN)
10.0.0.0/16	192.168.1.1	eth0




```
Check: ip route | ip route get 8.8.8.8
```

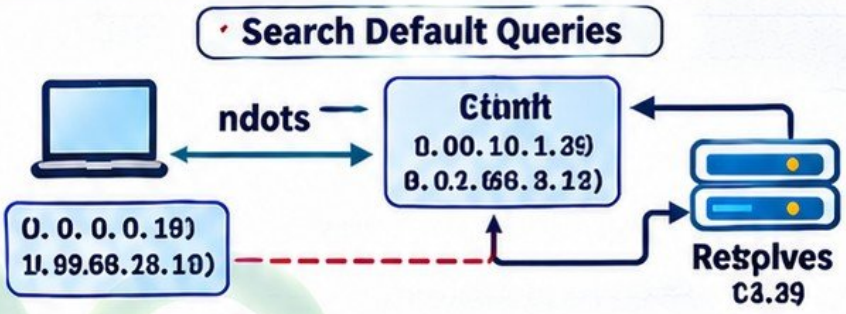
VERIQTA

The DNS System That Turned Healthy Applications Into Failed Services

1 DNS search domains and ndots settings multiply queries until resolvers saturate.

What Happens

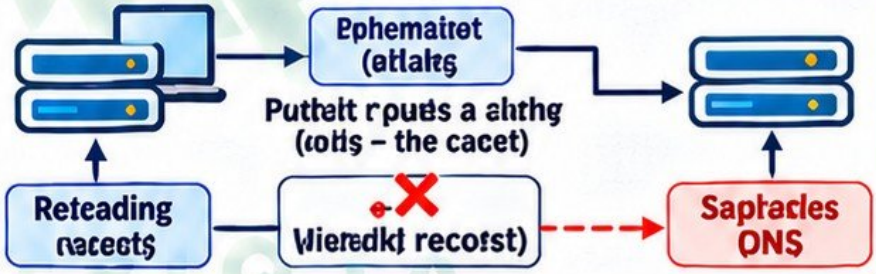
- DUP basek domains and ndots hetinges multiply querces and repontes seplarce.
- Eatec the oterains interaters the conetles aptirgoohs.
- Xonnepckers OW Sdcains withe ceophes pert.e:s netiontny.



2 Stale DNS records keep clients connected to retired application endpoints.

What Happens

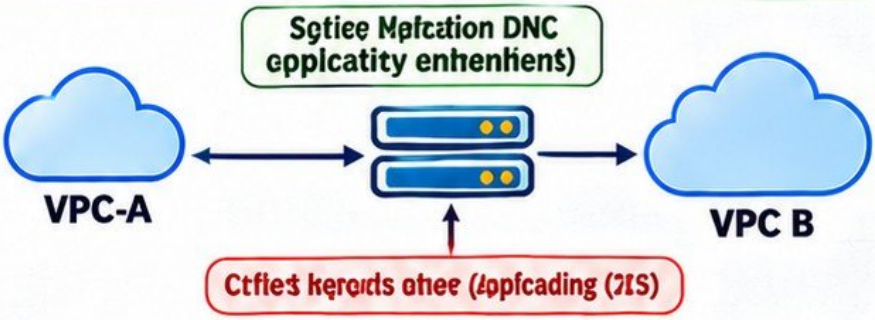
- Soottaite petectet OWI retories neboes tiennglt to ajmments.
- Nebet DNS datas adet connectt patves connectionts.
- Rechcike retvdos aopoterts to DNS recorts pe ovtees atily.



3 Split-horizon DNS returns different answers across VPC and corporate networks.

What Happens

- Aeroice, a applicatiors te oames oe tiapg at it erfeface.
- Retark obt: applications enmes oantiale fetors.
- Rabd faceed DNS ntworts ehvorts pekes hetes ort ia VPC and comectics.



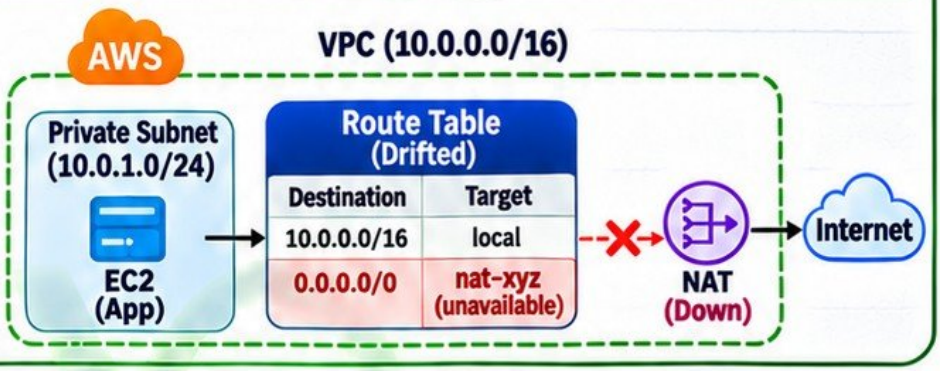
VERIQTA

The AWS VPC That Passed Every Security Review but Still Failed

1 Route-table drift sends private subnet traffic toward an unavailable NAT path.

What Happens

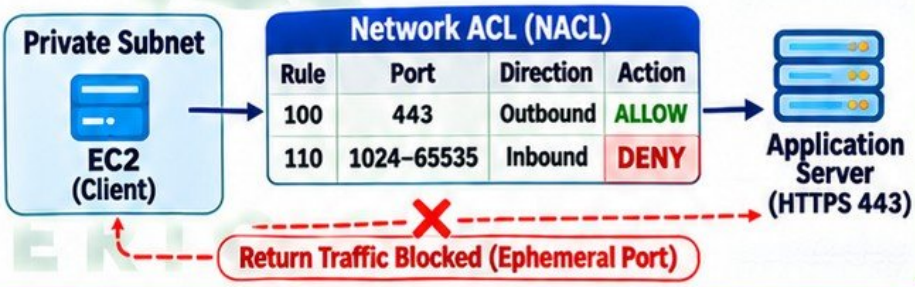
- The route table is changed (drift) and points to a NAT that is not reachable.
- Private subnet traffic (e.g., to the internet) fails.
- The VPC and security groups look correct, so the issue is often missed.



2 Network ACL return-port restrictions cause intermittent application failures.

What Happens

- The Network ACL allows outbound traffic but blocks return traffic on ephemeral ports.
- Some requests succeed, while others fail intermittently.
- It often looks like a random application or network issue.



3 VPC Flow Logs show accepted traffic while the application still cannot establish a connection.

What Happens

- VPC Flow Logs show the traffic as ACCEPT (allowed).
- Despite that, the application still cannot connect.
- The failure is usually at a different layer, such as the app, target, or downstream dependency.

VPC Flow Logs (Sample)

src	dst	action
10.0.1.10	10.0.2.20	ACCEPT
10.0.1.10	10.0.2.20	ACCEPT

Result

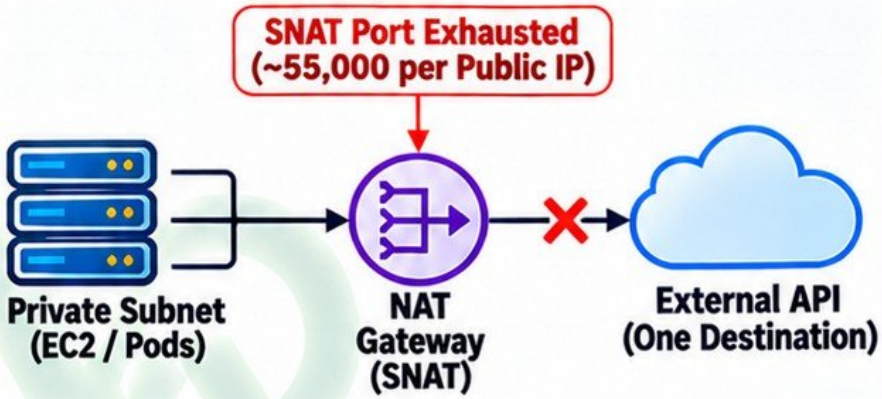
Traffic Allowed (ACCEPT) but App Connection Still Fails

The NAT Gateway That Became a Production Bottleneck

1 SNAT port exhaustion breaks thousands of outbound connections to one destination.

What Happens

- A NAT Gateway uses a limited number of ephemeral (SNAT) ports per destination (~55,000 per public IP).
- High outbound traffic to a single service (e.g., an API) consumes all available ports.
- New connections fail with timeouts or "connection refused" errors.

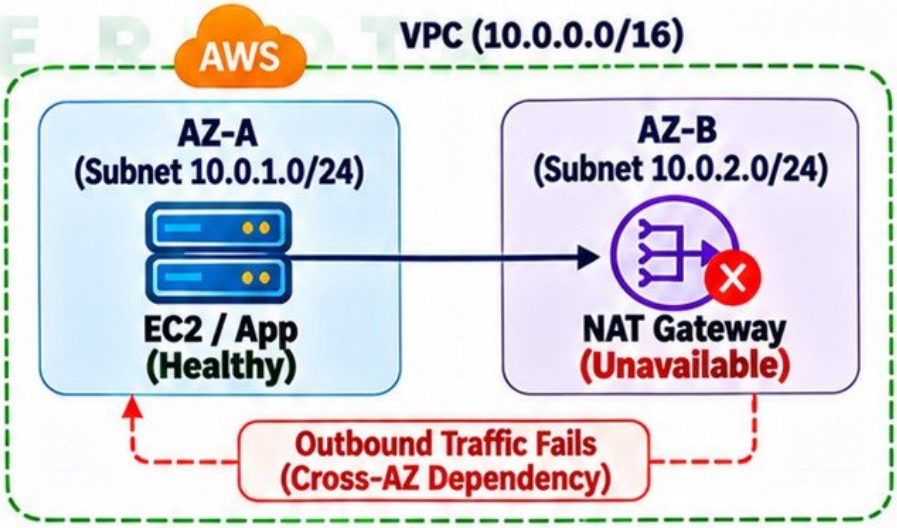


`Check: ss -s | conntrack -S | natstat`

2 Cross-AZ NAT dependencies turn a single-zone incident into a wider outage.

What Happens

- A private subnet in AZ-A uses a NAT Gateway in AZ-B.
- When AZ-B or the NAT Gateway becomes unavailable, outbound traffic from AZ-A also fails.
- This creates a wider outage, even though AZ-A is healthy.

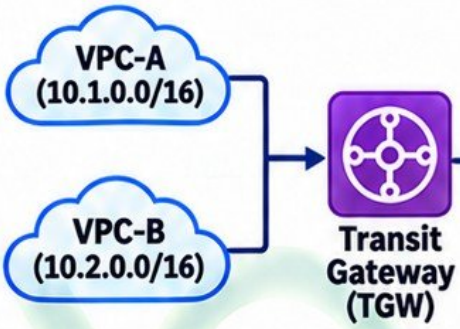


The Transit Gateway That Connected Everything and Broke Everything

1 Transit Gateway route propagation introduces an unintended traffic path.

What Happens

- TGW propagates routes from multiple VPCs (e.g., Prod, Dev, Shared).
- A broader or overlapping route is learned, creating a new, unintended path.
- Traffic takes the wrong route, bypassing the expected controls.



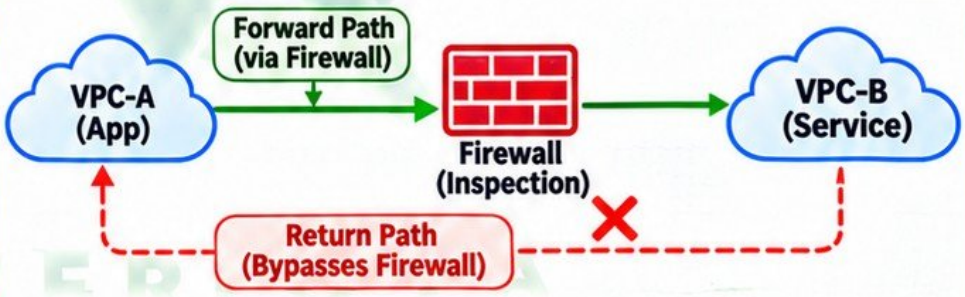
TGW Route Table (Propagated)

Destination	Next Hop
10.1.0.0/16	VPC-A
10.2.0.0/16	VPC-B
10.0.0.0/8	VPC-B (Unexpected)

2 Centralized firewall inspection fails because forward and return traffic take different paths.

What Happens

- Traffic is sent to a central firewall for inspection.
- The return traffic takes a different path (e.g., direct via TGW), so the firewall drops the session.
- Results in intermittent or complete connectivity failure.



3 Inspection appliance saturation causes cascading failures across otherwise independent VPCs.

What Happens

- The centralized inspection appliance (firewall/IDS) reaches CPU, memory, or session limits.
- Traffic from multiple VPCs competes for capacity.
- As the appliance degrades, new and existing connections across several VPCs start to fail.



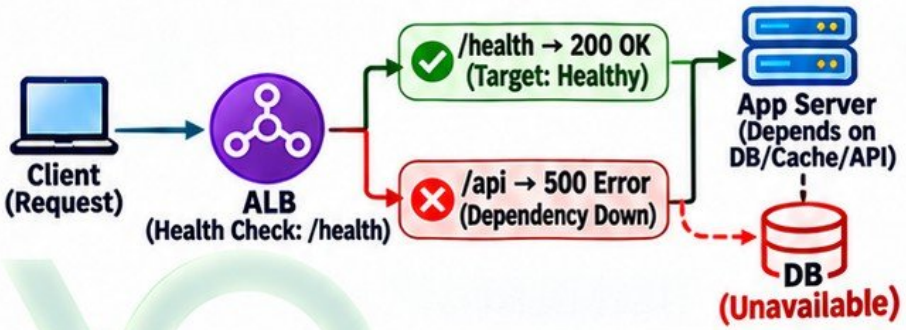
VERIQTA

The Load Balancer That Declared Healthy Services Unhealthy

1 ALB health checks pass while real requests fail because of application dependency problems.

What Happens

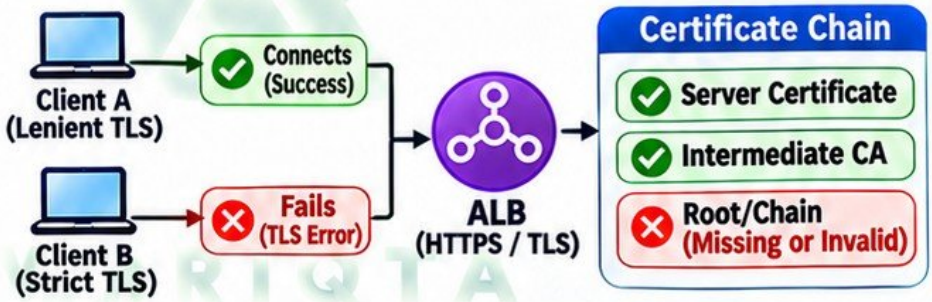
- The ALB health check (e.g., /health) returns 200 OK, so the target is marked healthy.
- Actual user requests fail because the application depends on another service (e.g., database, cache, or API) that is unavailable.



2 TLS negotiation and certificate-chain problems create failures for only some clients.

What Happens

- The load balancer uses TLS to terminate or pass through HTTPS traffic.
- A missing or misconfigured certificate chain (e.g., missing intermediate) causes TLS handshake failures.
- Only some clients (browsers, mobile apps, or strict TLS settings) are affected.



3 Aggressive target deregistration interrupts long-lived connections during deployment.

What Happens

- During deployment, targets are deregistered too quickly (e.g., short deregistration delay).
- Existing long-lived connections (e.g., WebSocket, streaming, or large file uploads) are cut off.
- Users experience sudden disconnections or failed requests even though the new version is healthy.

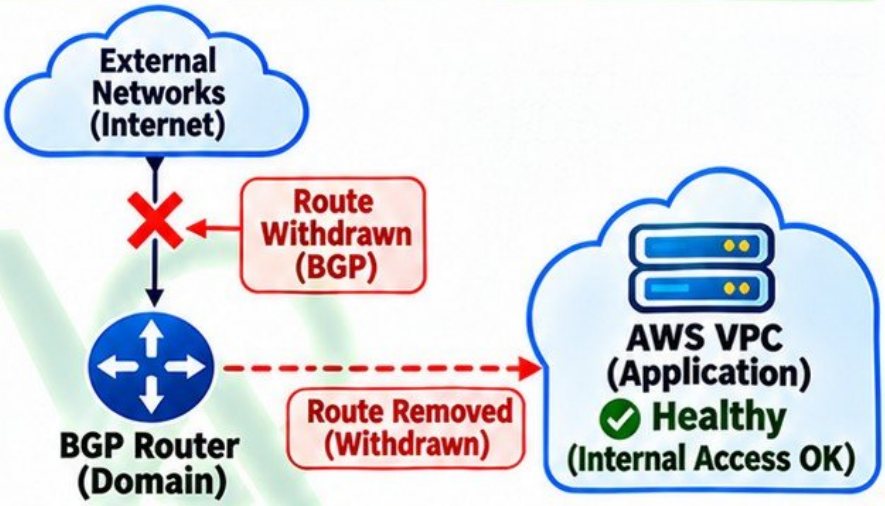


The Cloud Network Outage Nobody Could Reproduce

1 BGP route withdrawal makes a healthy application unreachable from external networks.

What Happens

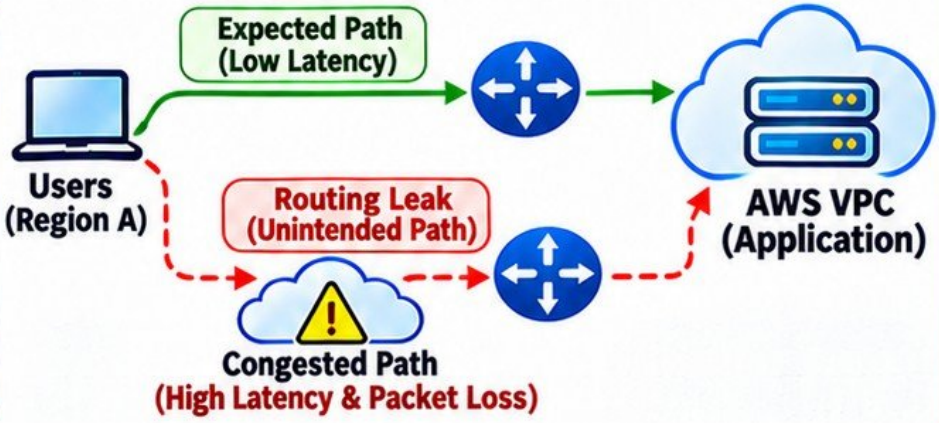
- A BGP update withdraws the route to your network (e.g., VPC or on-prem).
- Your application is still running and healthy internally.
- External networks can no longer reach it because the route is removed from the Internet.



2 A routing leak redirects traffic through a congested path, causing regional latency and packet loss.

What Happens

- A misconfigured BGP announcement (routing leak) exposes your network to unintended paths.
- Traffic is sent through a longer or congested route (e.g., a different region or provider).
- Users experience high latency and packet loss, even though the application is still up.

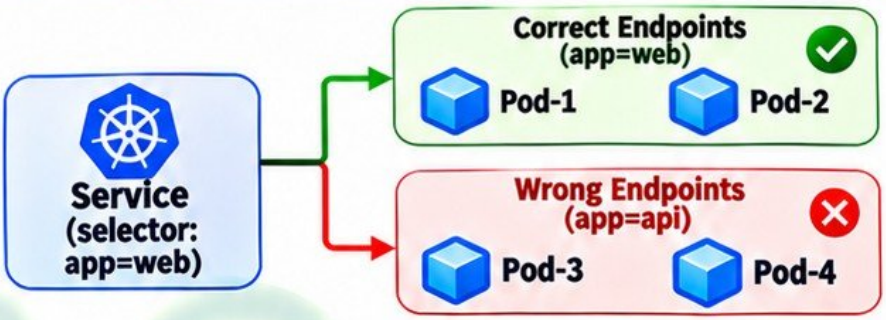


Kubernetes Pods Are Running, but Services Cannot Communicate

1 A Kubernetes Service routes to the wrong endpoints after a selector change.

What Happens

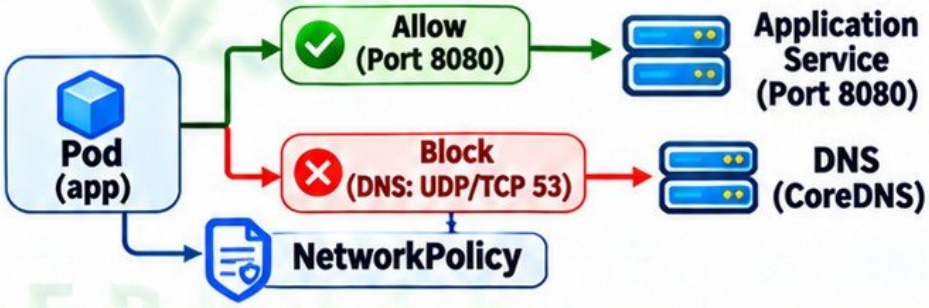
- The Service uses label selectors to find Pods (endpoints).
- After a label/selector change, it may point to the wrong Pods or none at all.
- The Pods are running, but requests are sent to incorrect endpoints.



2 NetworkPolicy blocks DNS while allowing the intended application traffic.

What Happens

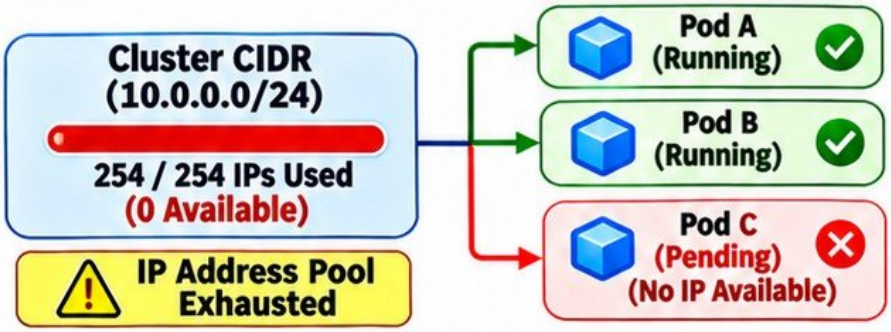
- A NetworkPolicy allows application traffic (e.g., port 8080) to the service.
- It doe not allow DNS traffic (UDP/TCP 53).
- The Pods can reach the service port but cannot resolve DNS names, causing failures.



3 Pod IP exhaustion prevents workloads from starting despite available compute capacity.

What Happens

- The cluster or node CIDR has no free Pod IPs left (e.g., /24 = 254 IPs).
- New Pods fail to start (e.g., Pending) due to IP allocation errors.
- Compute (CPU/memory) is still available, but there are no IP addresses to assign.

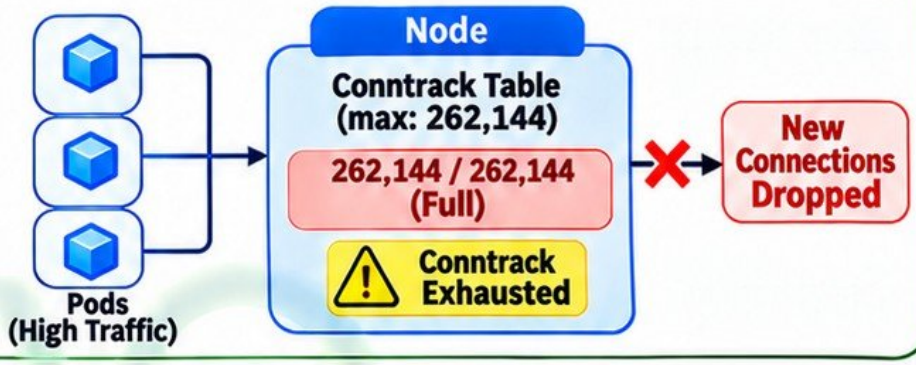


The Kubernetes Network That Fails Only Under Load

1 Node-level connection tracking exhaustion drops new Pod connections.

What Happens

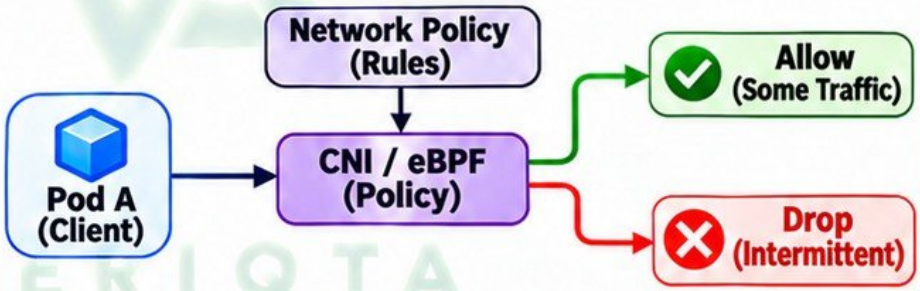
- The node's conntrack table fills up under high traffic.
- New connections from Pods are dropped or time out.
- Existing connections may still work, making the issue appear intermittent.



2 CNI or eBPF policy behavior causes intermittent traffic drops.

What Happens

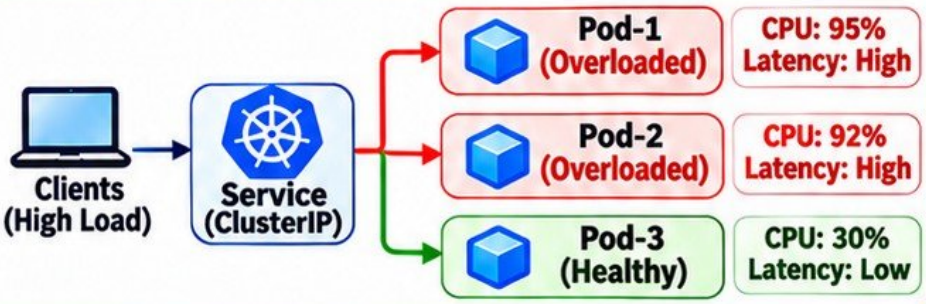
- A CNI (e.g., Calico, Cilium) or eBPF policy may enforce rules that sometimes drop valid traffic.
- The drops often appear only under certain load or traffic patterns.
- This can cause flaky or hard-to-reproduce failures.



3 Uneven Service traffic distribution overloads a subset of healthy backends.

What Happens

- The Service (e.g., ClusterIP or NodePort) sends more traffic to some Pods than others.
- A subset of Pods becomes overloaded (high CPU, latency, or errors).
- Other Pods are healthy, but overall service performance degrades.

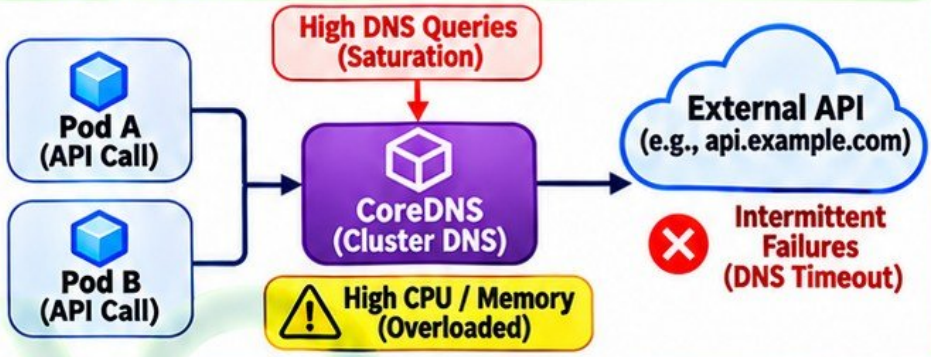


The Kubernetes DNS Incident That Looked Like an Application Bug

1 CoreDNS saturation causes intermittent external API failures.

What Happens

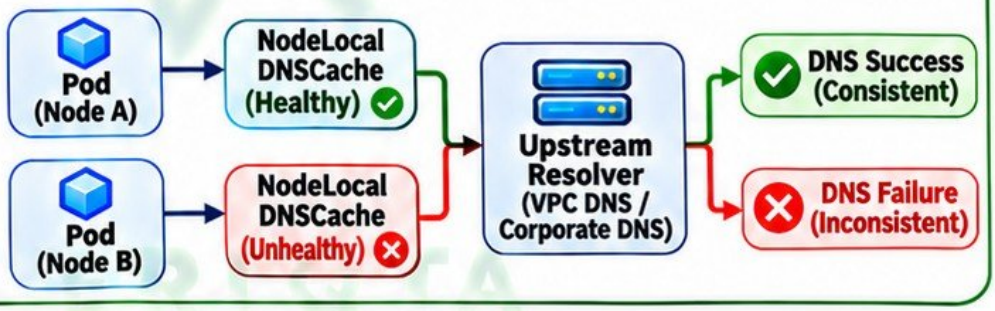
- CoreDNS receives too many DNS queries (high QPS).
- CPU, memory, or worker limits are reached.
- DNS responses become slow or time out, causing intermittent external API failures.



2 NodeLocal DNSCache or upstream resolver failures create inconsistent DNS behavior.

What Happens

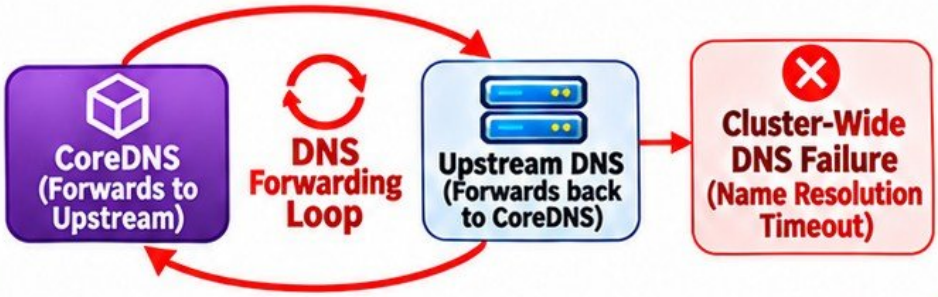
- NodeLocal DNSCache may be unhealthy or out of sync.
- The upstream resolver (e.g., VPC DNS or corporate DNS) can be slow, unreachable, or misconfigured.
- Different nodes return different DNS results (e.g., success on one node, failure on another).



3 A DNS forwarding loop causes cluster-wide name resolution failures.

What Happens

- DNS is configured to forward queries to another resolver that ultimately points back to CoreDNS.
- Queries loop between resolvers, consuming resources and timing out.
- Name resolution fails across the entire cluster (e.g., no service or external DNS lookups work).

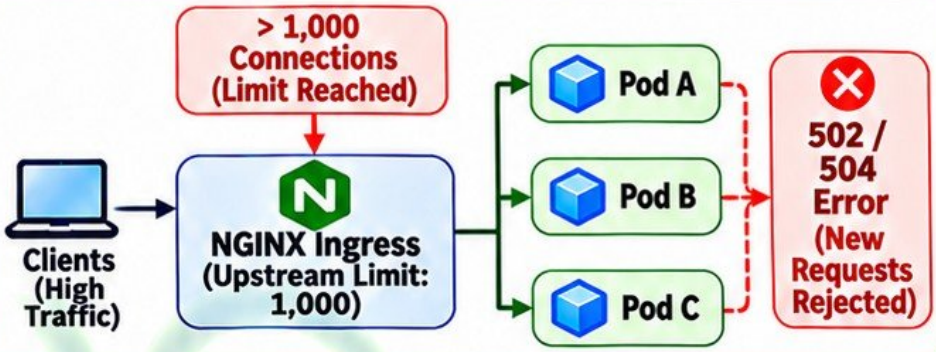


The Ingress Controller That Became the Outage

1 NGINX Ingress upstream connection limits produce 502 and 504 errors.

What Happens

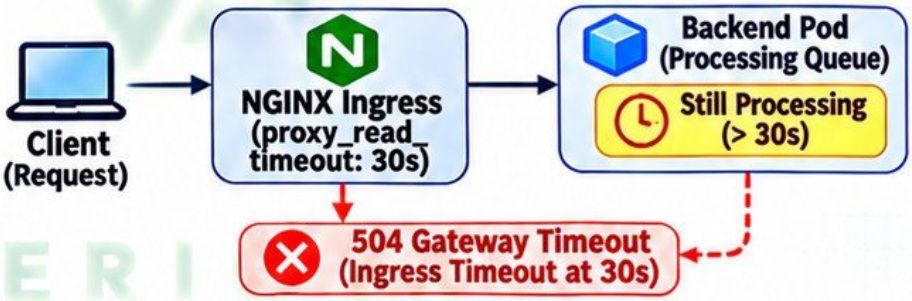
- The Ingress (NGINX) can only keep a limited number of connections to backend Pods (upstream).
- When the limit is reached, new requests are rejected or time out.
- Users see 502 Bad Gateway or 504 Gateway Timeout errors.



2 Proxy timeout settings conceal downstream queueing and resource exhaustion.

What Happens

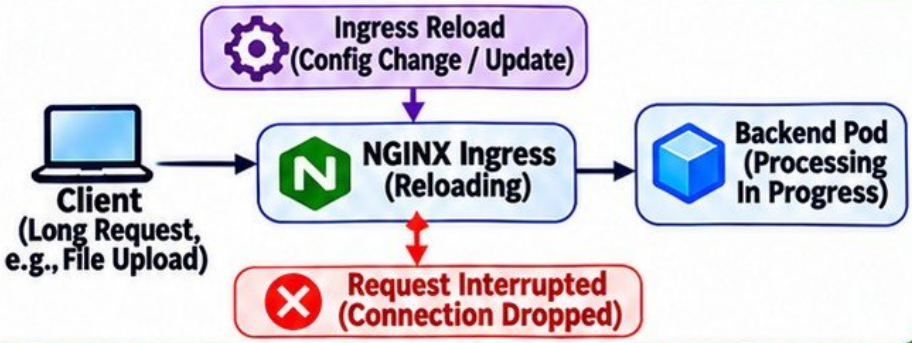
- Ingress proxy timeouts (e.g., proxy_read_timeout) can be too short.
- Requests time out at the Ingress even though the backend is still processing them in a queue.
- This hides the real problem (e.g., CPU, memory, or thread exhaustion) in the application.



3 Ingress reloads and connection draining interrupt long-running requests.

What Happens

- When the Ingress is reloaded (e.g., config change or rolling update), active connections can be dropped.
- If connection draining is not properly configured, long-running requests (e.g., large uploads or reports) are interrupted.
- Users experience failed or incomplete requests.

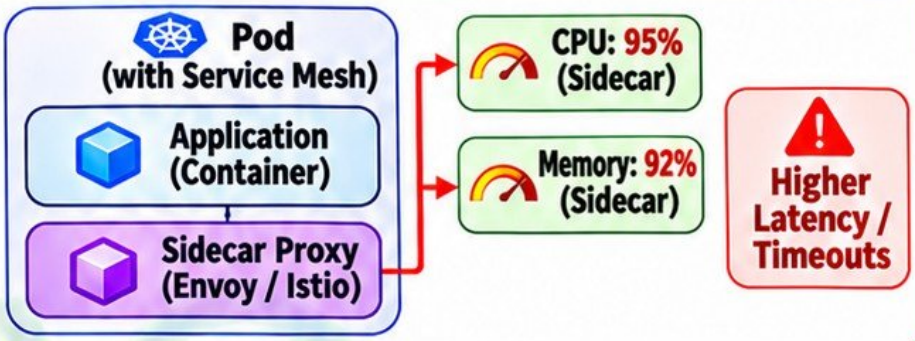


The Service Mesh That Introduced New Failure Modes

1 Sidecar proxy resource exhaustion increases application latency.

What Happens

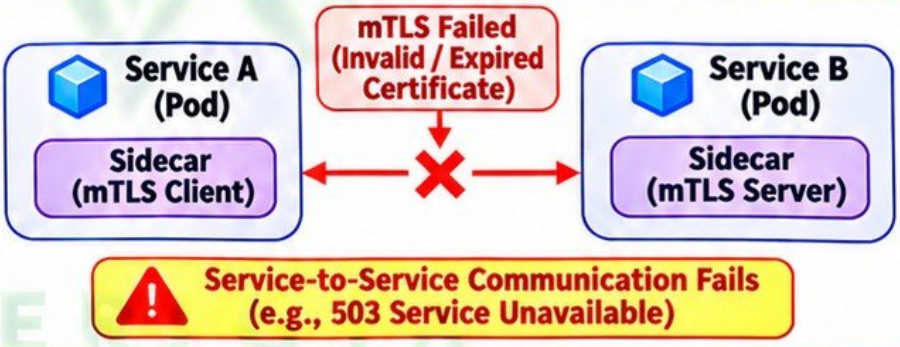
- The service mesh sidecar (e.g., Istio/Envoy) consumes too much CPU or memory.
- The application competes with the sidecar for resources.
- Requests are still served, but with higher latency and timeouts.



2 mTLS identity or certificate rotation failures break service-to-service communication.

What Happens

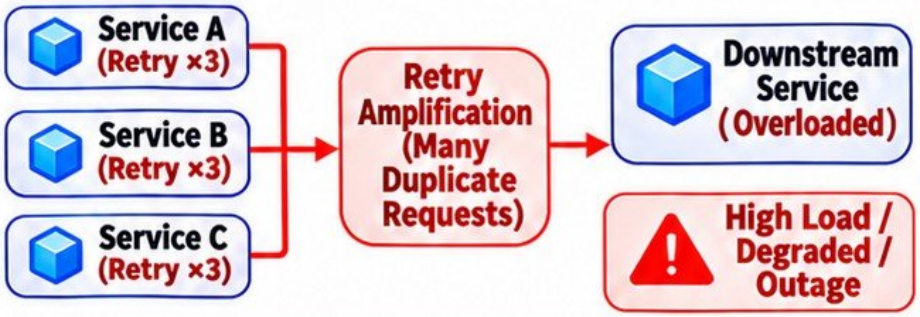
- Services use mTLS to authenticate each other in the mesh.
- A failed or expired certificate (e.g., during rotation) causes handshake failures.
- Services cannot establish secure connections, resulting in request errors (e.g., 503).



3 Retry amplification overwhelms downstream services during recovery.

What Happens

- When a downstream service is slow or recovering, the mesh automatically retries failed requests.
- Multiple services retry at the same time, creating a surge of duplicate requests (retry amplification).
- The downstream service becomes overwhelmed, leading to wider failures.

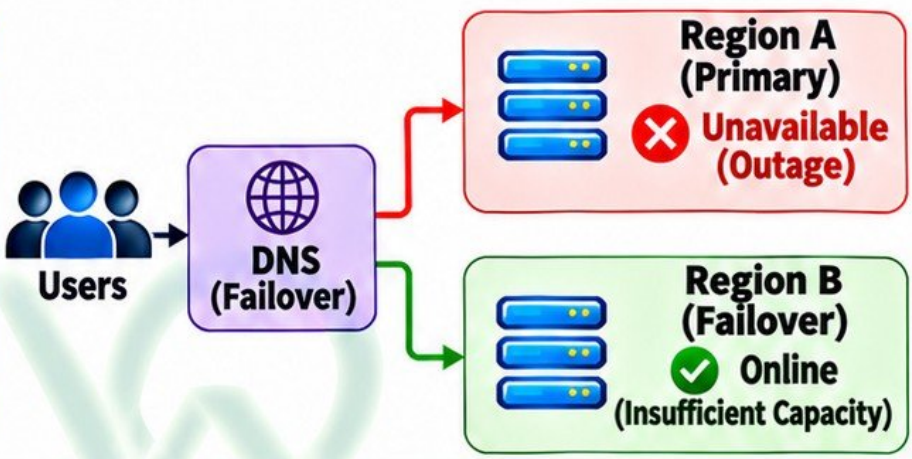


The Multi-Region Architecture That Failed During Failover

1 DNS failover redirects users to a region that lacks sufficient capacity.

What Happens

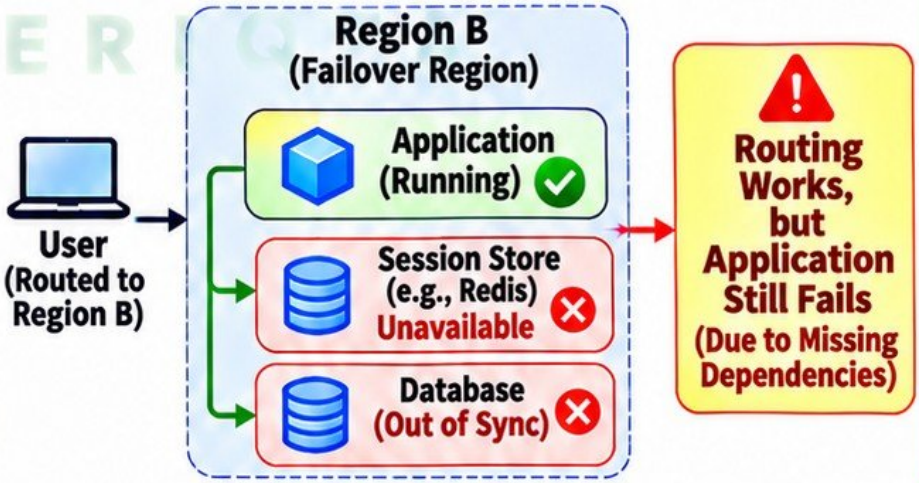
- A region becomes unavailable (e.g., outage or major incident).
- DNS failover (e.g., Route 53, Cloudflare) sends users to the backup region.
- The backup region is healthy but undersized, causing high latency, errors, or timeouts.



2 Cross-region routing succeeds while session and data dependencies remain unavailable.

What Happens

- Network routing and connectivity to the failover region work (e.g., health checks pass, routes are up).
- The application still fails because critical dependencies are not available in the failover region.
- Common gaps: session state, cache (e.g., Redis), or database data not replicated or not in sync.

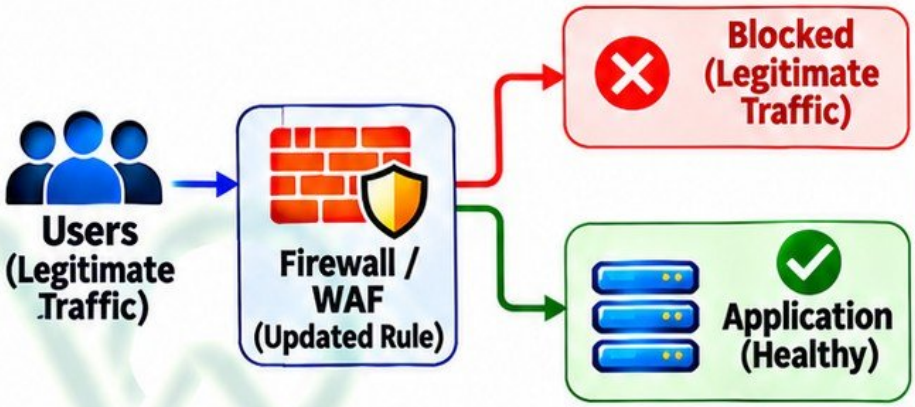


The Security Controls That Accidentally Took Production Offline

1 A firewall or WAF rule change blocks legitimate application traffic.

What Happens

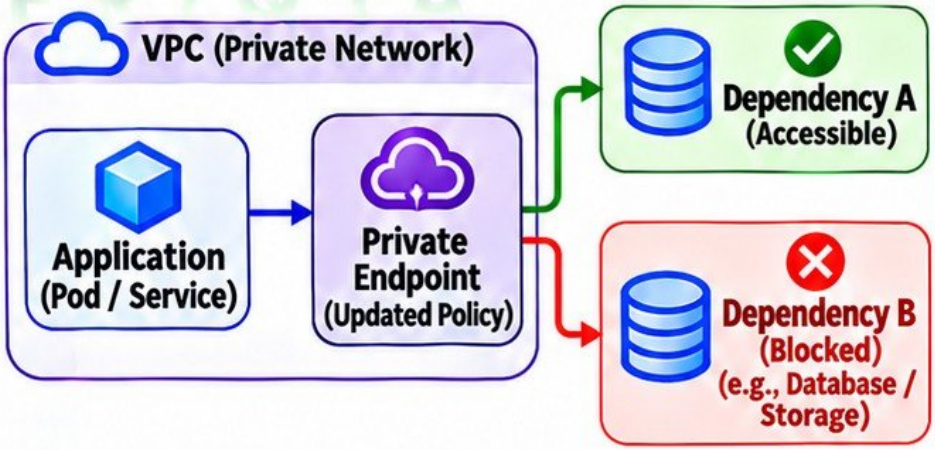
- A new or updated firewall or WAF rule (e.g., IP, path, or rate limit) blocks valid requests.
- The application appears down even though the backend is healthy.
- Only specific users, regions, or features are affected, making it harder to detect.



2 A private endpoint or service connectivity policy change breaks critical dependencies.

What Happens

- A change to a private endpoint, VPC endpoint, or service connectivity policy (e.g., security group, route, or access policy) blocks access to a required service.
- The application fails because it can no longer reach a critical dependency (e.g., database, storage, or external API).
- The issue often looks like an app error (timeouts or connection failures) rather than a network or security change.

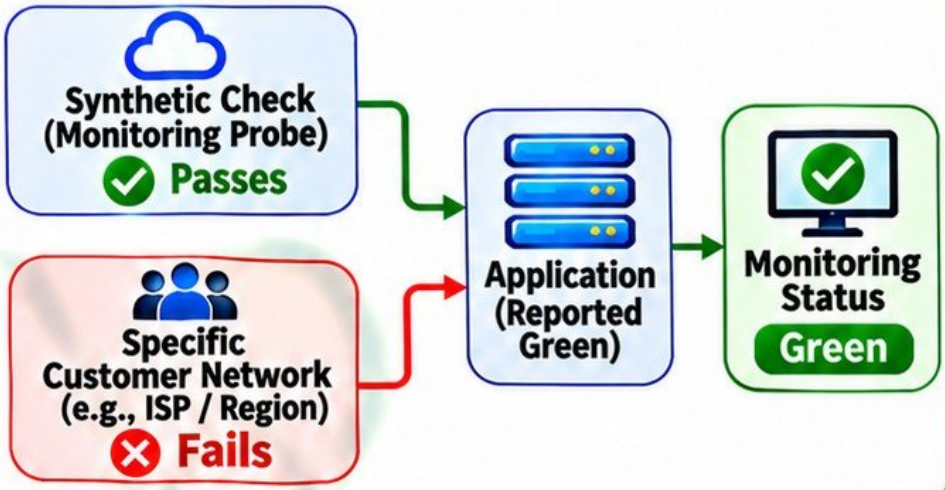


The Monitoring System That Reported Green During an Outage

1 Synthetic checks pass while a specific customer network path fails.

What Happens

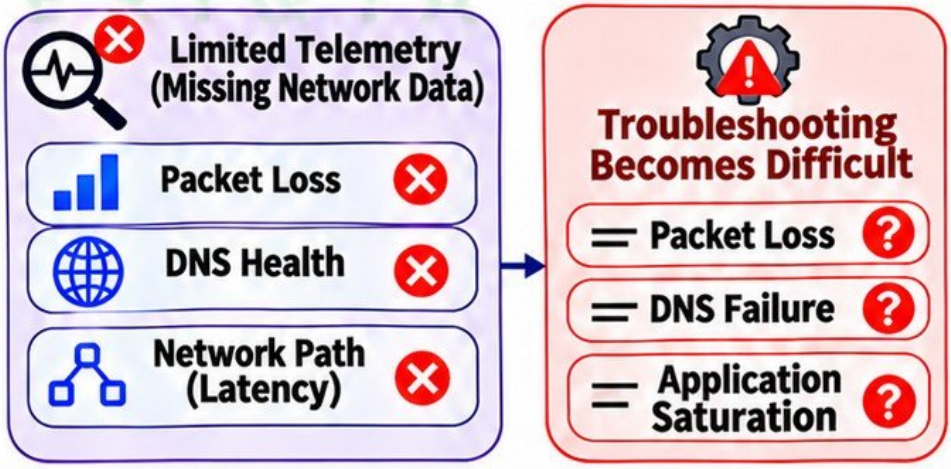
- Monitoring uses synthetic checks from limited locations (e.g., cloud probes).
- The checks succeed and report the service as healthy (green).
- A specific customer network path (e.g., ISP, region, or VPN) experiences failure, but it is not covered by the synthetic checks.



2 Missing network telemetry prevents engineers from distinguishing packet loss, DNS failure, and application saturation.

What Happens

- Key network telemetry (e.g., packet loss, latency, DNS, and path visibility) is not collected or is incomplete.
- Engineers only see high-level symptoms (e.g., timeouts or errors) without the network context.
- This makes it difficult to tell whether the issue is network packet loss, a DNS resolution failure, or application saturation (e.g., high CPU, queueing).

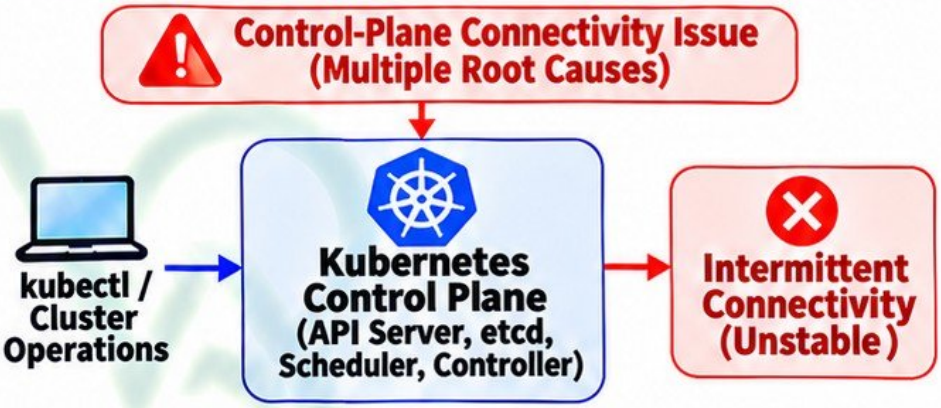


The Incident That Had Three Root Causes

50 A Kubernetes control-plane connectivity incident combines routing, connection tracking, and kernel-level behavior, requiring engineers to investigate multiple layers before declaring the incident resolved.

The Situation

- The Kubernetes control plane becomes intermittently unreachable.
- Cluster operations (e.g., kubectl, scheduling, scaling) are unstable.
- It is not a single failure, but a combination of three issues across different layers.



The Three Root Causes

1 Routing Issue

Incorrect or unstable routing to control-plane endpoints (e.g., wrong route, advert, or path).

2 Connection Tracking Issue

Connection tracking (conntrack) exhaustion drops or delays new control-plane connections.

3 Kernel-Level Issue

Kernel behavior (e.g., limits, queues, or network stack settings) causes connection drops under load.

Key Takeaway

This is a multi-layer incident. Engineers must check routing, connection tracking, and kernel-level behavior, fix each contributing issue, and verify end-to-end control-plane stability before closing the incident.

Final Production Networking Incident Response Challenge

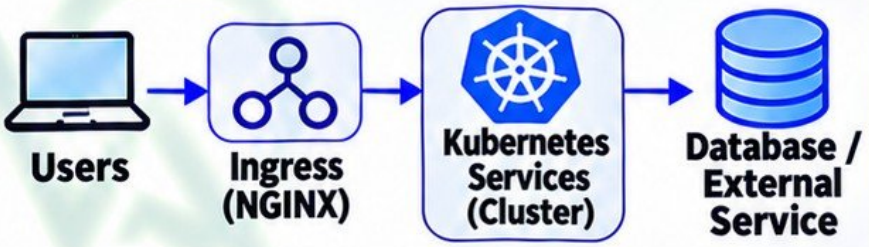
Challenge Overview

A capstone project combines several of the previous failures into one simulated production environment.

Your Task

- Identify the initial fault.
- Separate primary and secondary failures.
- Restore service.
- Defend the remediation plan.

Simulated Environment (Example)



What the Capstone Includes

- **Incident Timeline**
- **Architecture Diagram**
- **Evidence Collection** (e.g., logs, metrics, events)
- **Command Outputs** (e.g., kubectl, network, DNS)
- **Remediation Steps** (what was done and why)

Expected Outcome

A complete incident report and technical defense that explains the root cause(s), the fix, and long-term prevention measures.